

Plano de Ação — Determinismo e Redução de Custo de Token no Pipeline

Status: planejado (implementação incremental em feature/determinismo-pipeline-custos)

Data: 2026-08-13 **Escopo:** mapear, campo a campo, o que em analista-insumos → diretor-de-arte → redator-* → revisor-marca é decisão estrutural fixa (pode virar script/config) vs. criação de conteúdo real (tem que continuar em LLM), com base em evidência real de custo por etapa de um projeto de teste já executado.

1. Evidência real de custo (não é estimativa)

Rodada real (/produzir-comunicacao-completa, projeto zz-teste-painel-view, só material textos, via painel headless) já executada e com transcript completo salvo localmente. Tokens extraídos direto dos arquivos .jsonl de sessão (soma de input_tokens + output_tokens + cache_creation_input_tokens — exclui cache_read_input_tokens, que é cobrado a preço reduzido):

Etapa	Tokens “pagos”	% do total	Turnos	O que faz
Orquestrador principal (analista-insumos + diretor-de-arte + dispatch)	175.637	42,4%	68	Lê insumo, gera dossiê+briefing, despacha os 2 subagentes abaixo
subagente-revisor-marca	141.131	34,1%	33	Audita fidelidade à fonte/marca do material já escrito
subagente-produtor-textos	97.121	23,5%	18	Escreve os 3 textos (WhatsApp/Instagram/LinkedIn)
Total pago	413.889	100%	119	
(cache_read, cobrado a preço reduzido)	7.765.777	—	—	

Achado principal, contraintuitivo: a etapa de **revisão** (revisor-marca) custou mais token pago do que a etapa de **escrita** do conteúdo (produtor-textos) — 141k vs. 97k. Auditar custou mais caro que criar. Isso muda onde vale mais a pena investir em determinismo: não é só “a escrita é cara”, é “a auditoria é ainda mais cara”.

2. Mapeamento campo a campo (lido direto dos SKILL.md, não por memória)

analista-insumos

Campo/decisão	Tipo	Observação
Extrair fatos/claims do texto-base	Criativo/compreensão	Texto livre do cliente — não determinizável
Confirmar existência de cada imagem em disco	Estrutural, já determinizável	É um <code>Path.exists()</code> — hoje a LLM confere isso lendo, poderia vir pré-computado
Registrar <code>publico_alvo/objetivo_tom</code> do operador	Fixo (cópia, não decisão)	Já é só transcrição de <code>config_projeto.json</code> ; a skill já proíbe re-derivar
Implicações práticas do público/tom pros redatores	Criativo/interpretação	Não determinizável — depende do produto

diretor-de-arte

Campo/decisão	Tipo	Observação
Mensagem central (≤10 palavras)	Criativo	Não determinizável
Hierarquia de conteúdo	Criativo	Não determinizável
angulos_criativos (arte), angulos_por_tom (kits)	Criativo, mas com moldura fixa	Os 5 tons dos kits já são fixos (brand/tons-kit.json); só o conteúdo de cada ângulo é criativo
Estruturas dos presets /kit-completo-<publico>	Fixo	Já vem de tabela em SPEC_COMANDOS.md, a skill só aplica
Decompor objetivo_tom em objetivo+tom_de_voz	100% fixo — São todos os canais de fixos (educacional_comercial → educacional/comercial, etc.), escritos por extenso no próprio SKILL.md. Hoje a LLM faz uma “cópia” de uma tabela de 3 entradas que já está no seu próprio prompt — moveríamos isso pra fora, ganho pequeno mas real e zero risco	
mapeamento_por_material.textos.canais	Provavelmente redundante	redator-textos já escreve sempre os 3 canais (what-sapp/instagram/linkedin) por definição própria da skill — o diretor não precisaria decidir isso pra “textos”

redator-* (todos)

Campo/decisão	Tipo	Observação
Toda a prosa/copy	100% criativo	Núcleo de valor do produto — não determinizável
Limites de caractere, formato por canal	Já determinístico	Validado por <code>validar-textos.py/validar-dimensoes.py/validar-kit.py</code> , não pela LLM
CTA/assinatura dos kits	Já fixo	Vem de <code>brand/kits-conexao.json</code> , a LLM nem escreve isso
Hashtags “relevantes do nicho” (textos/Instagram)	Criativo, mas com risco documentado	A própria skill usa <code>#ConexaoImplantes</code> como <i>ex-emplo</i> no <code>SKILL.md</code> — foi exatamente isso que vazou por engano no nosso teste real e o revisor-marca teve que corrigir. Achado colateral: vale trocar o exemplo do skill por algo obviamente genérico/fictício, risco zero, fora do escopo de determinismo mas achado real

revisor-marca

Campo/decisão	Tipo	Observação
Rodar <code>validar-*.py</code> (cores, dimensões, PDF, HTML, kit)	Já determinístico	Scripts com exit code, a skill só interpreta o JSON
Logo presente	Já determinístico	<code>scripts/validar-logo.py</code> já existe e cobre isso
Transparência de imagem	Já determinístico	<code>scripts/validar-transparencia.py</code> já existe
Hex de cor fora do design system	Já determinístico	<code>scripts/validar-design-tokens.py</code> já existe
Fidelidade à fonte — “checagem factual linha a linha”	LLM hoje, mas a própria skill já descreve como mecânica	Maior oportunidade real: comparar cada claim/número/hashtag do material contra o dossiê é busca textual, não julgamento — dá pra pré-filtrar com regex e sobrar só os casos ambíguos pra LLM decidir
Aderência a público/tom (soa como “distribuidor”?)	Semântico	Não determinizável — precisa entender registro de linguagem
Julgamento de design (motion, componente decorativo vs. funcional)	Explicitamente qualitativo	A própria skill diz que isso funciona “em qualquer harness” por ser orientação embutida — não é candidato a script
Oportunidade de enriquecimento perdida (dado numérico sem gauge/donut)	Parcialmente determinizável	Detectar “há número/% no dossiê sem componente correspondente no HTML” é regex+grep; a decisão de <i>qual</i> componente cabe continua sendo julgamento

3. Plano de ação (2 melhorias, por ordem de risco/retorno)

3.1. Baixo risco, baixo retorno — decomposição fixa de `objetivo_tom`

Mover a tabela de 3 entradas (já escrita por extenso no SKILL.md de diretor-de-arte) para `scripts/parametros_projeto.py`, como uma função pura testável. Ganho pequeno (a LLM já tinha a tabela no próprio prompt, não precisava “pensar” muito) — mas é grátis e zero risco de regressão de qualidade, então entra primeiro.

3.2. Maior retorno — pré-filtro determinístico de claims antes do revisor-marca

Novo script `scripts/extraair-claims-candidatos.py`: - Lê os `.txt` do material (`output/<slug>/<tipo>/*.txt`) e `dossie_insumos.md`. - Extrai candidatos via regex: números com unidade, percentuais, hashtags, datas, sequências de palavras capitalizadas (*proprio-noun-ish*). - Para cada candidato, verifica se aparece (normalizado) no dossiê ou numa allowlist de marca — se não aparecer, marca como **candidato a verificar**. - Grava `output/<slug>/revisao/candidatos_verificacao.json` — nunca falha com exit 1 (é assistivo, não um gate), e nunca substitui a checagem semântica completa do revisor-marca (paráfrase que muda o sentido não aparece em regex). - revisor-marca passa a rodar esse script **antes** da checagem manual da fonte e usa a lista como checklist inicial — reduz quanto ele precisa “procurar do zero”.

Isso ataca diretamente a etapa que a evidência real mostrou ser a mais cara (34,1% do total, mais cara que a própria escrita do conteúdo).

4. Como vamos medir se funcionou

1. **Baseline já existe** (seção 1 acima) — não precisa re-rodar pra ter o “antes”.
2. Depois de implementar 3.1 e 3.2, rodar `/produzir-comunicacao-completa` de novo, com o **mesmo insumo/material** (textos), slug novo.
3. Extrair os mesmos números (tokens pagos por etapa) do transcript da rodada nova.
4. Comparar lado a lado — orquestrador, produtor, revisor — e montar um comparativo visual (antes/depois) pra decidir se a economia compensa o esforço de manutenção extra (mais um script pra manter).

5. Reversibilidade

Tudo em `feature/determinismo-pipeline-custos`, sem alterar `main`. Se a comparação antes/depois não mostrar economia real, é só não fazer merge.